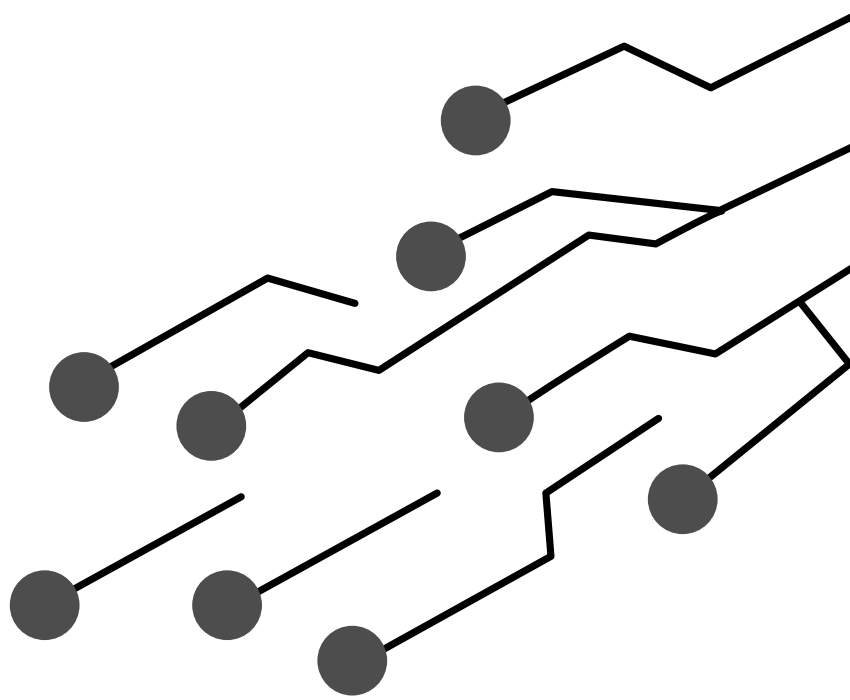
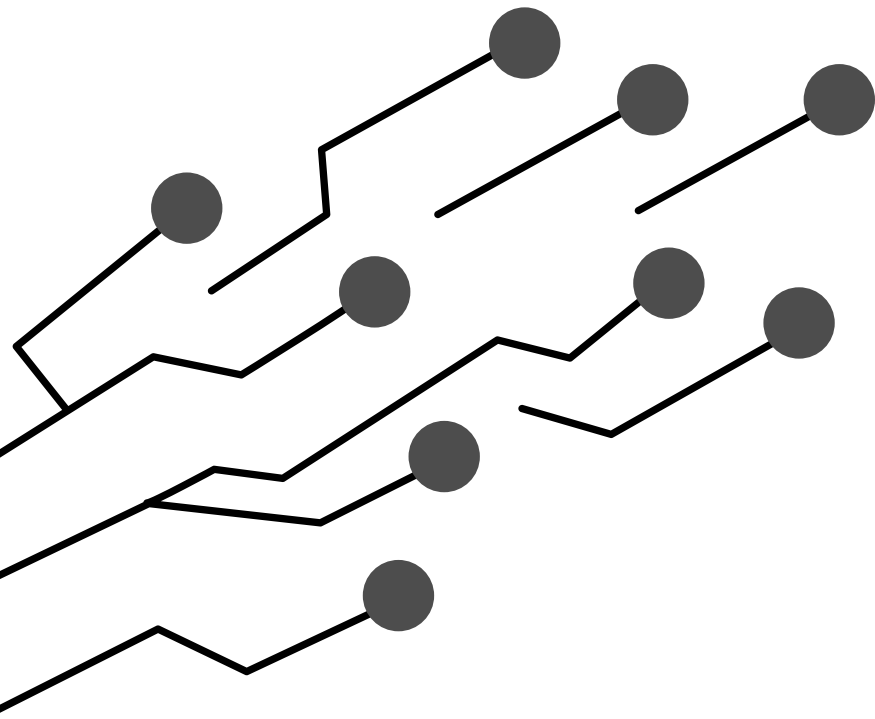


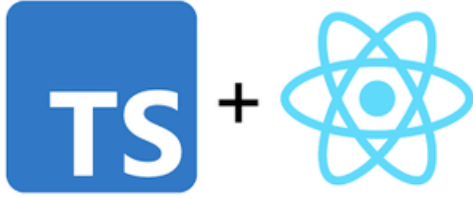


Technology Requirements



Technology Requirements

Frontend & Web Dashboard



React is widely adopted and provides an efficient way to build interactive user interfaces through reusable components. This is particularly useful for developing dynamic dashboards such as the placement recommendation interface, where data updates frequently. TypeScript enhances code reliability by introducing static typing, allowing many errors to be detected during development rather than at runtime. This improves maintainability and reduces debugging effort. To support data visualisation and efficient data handling, libraries such as Recharts will be used to display scoring breakdowns and analytics, while TanStack Query will manage API communication, caching, and synchronization. These tools simplify development and improve performance when working with frequently updated data.



Since React will be used on the frontend, vercel will be used to host web applications

Backend API



NestJS is preferred due to its use of TypeScript, which aligns with the frontend technology stack and reduces the need for developers to switch between languages. It also provides a modern development experience with built-in support for modular design and testing.

Database & Caching



PostgreSQL is chosen for its reliability, strong support for structured data, and advanced features such as row-level security. This is particularly important for supporting multi tenant requirements and ensuring data isolation between different organisations.



Redis will be used as both a caching layer and a support system for background job processing. This improves performance and reduces the load on the primary database.

Infrastructure & Hosting



AWS allowing faster development and deployment to run the application in a cloud environment without incurring any costs

CI/CD



GitHub Actions

Continuous Integration and Deployment will be handled using GitHub Actions, enabling automated testing and deployment processes. Development will be carried out using Visual Studio Code, which offers strong support for TypeScript and a wide range of extensions that improve developer productivity.

Testing



Jest will be used as the primary testing framework for unit and integration testing. It provides a simple and efficient environment for testing business logic, such as the fit score matching engine, ensuring correctness and reliability of core system functionality. Supertest will be used to test API endpoints by simulating HTTP requests and verifying responses. This ensures that the REST API behaves as expected, including correct handling of data, validation, and role-based access control.



react-testing-library

React Testing Library will be used for frontend testing, focusing on user interactions and component behaviour rather than implementation details. This approach ensures that the user interface functions correctly from the user's perspective and improves maintainability of tests as the application evolves